

User Guide for Intel® Distribution for GDB*

Contents

Chapter 1: Overview

Debugging a Kernel on CPU and GPU: Major Differences	7
Hard-to-find and Useful Information for gdb-oneapi	9
Unique Features in Intel® Distribution for GDB*	13
Intel® Distribution for GDB* FAQ.....	15
Switch Between a CPU Context and a GPU Context When Debugging with VSCode*	19
Videos: Intel® Distribution for GDB*	20

Overview

1

NOTE Support for GDB IDE extensions for both Microsoft Visual Studio* and Microsoft Visual Studio Code* has been deprecated as of the Intel® oneAPI Toolkit 2025.2.0 release.

What is Intel® Distribution for GDB*?

The Intel® Distribution for GDB* is an application debugger that allows you to inspect and modify the program state. With the debugger, both the host part of your application and kernels that are offloaded to a device can be debugged seamlessly in the same debug session. The debugger supports CPU and GPU devices. Major features of the tool include:

- Automatically attaching to the GPU device to listen to debug events
- Automatically detecting JIT-compiled, or dynamically loaded, kernel code for debugging
- Defining breakpoints (both inside and outside of a kernel) to halt the execution of the program
- Listing the threads; switching the current thread context
- Listing SIMD lanes; switching the current SIMD lane context per thread
- Evaluating and printing the values of expressions in multiple thread and SIMD lane contexts
- Inspecting and changing register values
- Disassembling the machine instructions
- Displaying and navigating the function call-stack
- Source- and instruction-level stepping
- Non-stop and all-stop debug mode
- Recording the execution using Intel Processor Trace (CPU only)
- Printing of Intel PT PTWRITE payloads and asynchronous events in the instruction history and function-call history
- Reading and writing Intel® Advanced Matrix Extensions (Intel® AMX) registers
- Reading and writing of the Intel® CET Shadow Stack Pointer (pl3_ssp) register
- Reading and writing of the Intel® APX registers (Extended GPRs \$r16 - \$r31) including byte, word and dword pseudo registers

For more information and links to full documentation for Intel Distribution for GDB*, see [Get Started with Intel® Distribution for GDB* on a Linux* host](#) and [Get Started with Intel® Distribution for GDB* on a Windows* host](#).

Debug GPU Execution Using Intel® Distribution for GDB* on compatible GPUs

Intel® Distribution for GDB* is extensively documented in [Get Started with Intel® Distribution for GDB* on a Linux* host](#) and [Get Started with Intel® Distribution for GDB* on a Windows* host](#). Useful commands are briefly described in the [Intel® Distribution for GDB* Cheat Sheet](#). However, since debugging applications with GDB* on a GPU differs slightly from the process on a host (some commands are used differently and you might see some unfamiliar output), some of those differences are summarized here.

The [Debug Examples in Linux](#) shows sample debug sessions where we start a debug session of a SYCL program, define a breakpoint inside the kernel, run the program to offload to the GPU, print the value of a local variable, switch to the SIMD lane 5 of the current thread, and print the variable again.

As in normal GDB*, for a command <CMD>, use the `help <CMD>` command of GDB to read the information text for <CMD>. For example:

```
(gdb) help info threads
Display currently known threads.
Usage: info threads [OPTION]... [ID]...
If ID is given, it is a space-separated list of IDs of threads to display.
Otherwise, all threads are displayed.

Options:
  -gid
    Show global thread IDs.

  -stopped
    Show stopped threads only.
```

Inferiors, Threads, and SIMD Lanes Referencing in GDB*

The threads of the application can be listed using the debugger. The printed information includes the thread ids and the locations that the threads are currently stopped at. For the GPU threads, the debugger also prints the active SIMD lanes.

In the example referenced above, you may see some unfamiliar formatting used when threads are displayed via the GDB `info threads -stopped` command:

```
(gdb) info threads -stopped
Id          Target Id          Frame
1.1         Thread 0x7ffff502ccc0 (LWP 124773) "array-transorm." 0x00008000001c3a10 in
clock_gettime ()
1.3         Thread 0x7fffe8e6e640 (LWP 124787) "array-transorm."
__futex_abstimed_wait_common64 (private=1, cancel=true, abstime=0x7fffe8e6dd30, op=137,
expected=0, futex_word=0x1bdaa60) at ./nptl/futex-internal.c:57
2.1:[0-15]   ZE 0.0.0.0          main::
{lambda(auto:1&)#1}::operator()<syctl::_V1::handler>(syctl::_V1::handler&) const::
{lambda(syctl::_V1::id<1>)#1}::operator() (syctl::_V1::id<1>) const (this=0xff000000002e0590,
index=syctl::id = 16) at array-transform.cpp:61
* 2.9:[*0 1-15] ZE 0.0.1.0          main::
{lambda(auto:1&)#1}::operator()<syctl::_V1::handler>(syctl::_V1::handler&) const::
{lambda(syctl::_V1::id<1>)#1}::operator() (syctl::_V1::id<1>) const (this=0xff000000002fc590,
index=syctl::id = 48) at array-transform.cpp:61
2.33:[0-15]   ZE 0.0.4.0          main::
{lambda(auto:1&)#1}::operator()<syctl::_V1::handler>(syctl::_V1::handler&) const::
{lambda(syctl::_V1::id<1>)#1}::operator() (syctl::_V1::id<1>) const (this=0xff00000000350590,
index=syctl::id = 0) at array-transform.cpp:61
2.41:[0-15]   ZE 0.0.5.0          main::
{lambda(auto:1&)#1}::operator()<syctl::_V1::handler>(syctl::_V1::handler&) const::
{lambda(syctl::_V1::id<1>)#1}::operator() (syctl::_V1::id<1>) const (this=0xff0000000036c590,
index=syctl::id = 32) at array-transform.cpp:61
```

Here, GDB is displaying the threads with the following format:

<inferior_number>.<thread_number>:<SIMD Lane/s>

So, for example, the thread id "2.33: [0-15]" refers to SIMD lanes 0, 1, ..., 15 of thread 33 running on inferior 2. In the thread id "2.9: [*0 1-15]", the selected lane 0 is additionally marked with an asterisk *.

An “inferior” in the GDB* terminology is the process that is being debugged. In the debug session of a program that offloads to the GPU, there will typically be two inferiors; one “native” inferior representing a host part of the program (inferior 1 above), and another “remote” inferior representing the GPU device (inferior 2 above). Intel® Distribution for GDB* automatically creates the GPU inferior - no extra steps are required.

When you print the value of an expression, the expression is evaluated in the context of the current thread’s current SIMD lane. You can switch the thread as well as the SIMD lane to change the context using the “thread” command such as “thread 3:4”, “thread :6”, or “thread 7”. The first command makes a switch to the thread 3 and SIMD lane 4. The second command switches to SIMD lane 6 within the current thread. The third command switches to thread 7. The default lane selected will either be the previously selected lane, if it is active, or the first active lane within the thread.

The `thread apply` command may be similarly broad or focused (which can make it easier to limit the output from, for example, a command to inspect a variable). For more details and examples about debugging with SIMD lanes, see [Debug Examples in Linux](#).

The `thread filter` command can be used to get a list of filtered thread ids using location and expression filter options. This helps to select and then debug the required threads (e.g., to inspect threads for a specific workitem local id). For more details and examples about debugging with SIMD lanes, see [Debug Examples in Linux](#).

More information about threads and inferiors in GDB can be found at <https://sourceware.org/gdb/current/onlinedocs/gdb.html/Threads.html> and <https://sourceware.org/gdb/current/onlinedocs/gdb.html/Inferiors-Connections-and-Programs.html>.

Controlling the Scheduler

By default, when a thread hits a breakpoint, the debugger stops all the threads before displaying the breakpoint hit event to the user. This is the all-stop mode of GDB. In the non-stop mode, the stop event of one or more threads is displayed while the other threads run freely.

In all-stop mode, when a thread is resumed (for example, to resume normally with the `continue` command, or for stepping with the `next` command), all the other threads are also resumed. If you have some breakpoints set in threaded applications, this can quickly get confusing, as the next thread that hits the breakpoint may not be the thread you are following.

You can control this behavior using the `set scheduler-locking` command to prevent resuming other threads when the current thread is resumed. This is useful to avoid intervention of other threads while only the current thread executes instructions. Type `help set scheduler-locking` for the available options, and see [Stopping and Starting Multi-thread Programs](#) for more information. Note that SIMD lanes cannot be resumed individually; they are resumed together with their underlying thread.

In non-stop mode, by default, only the current thread is resumed. To resume all threads, pass the “-a” flag to the `continue` command.

Dumping Information about One or More Threads/Lanes (Thread Apply)

Commands for inspecting the program state are typically executed in the context of the current thread’s current SIMD lane. Sometimes it is desired to inspect a value in multiple contexts. For such needs, the `thread apply` command can be used. For instance, the following executes the `print element` command for the SIMD lanes 3-5 of Thread 2.129:

```
(gdb) thread apply 2.129:3-5 print element
```

Similarly, the following runs the same command in the context of SIMD lane 3, 5, and 6 of the current thread:

```
(gdb) thread apply :3 :5 :6 print element
```

The `all-lanes` argument can be passed to the `thread apply` command to run the command on all active lanes:

```
(gdb) thread apply all-lanes print element
```

The `thread apply` command supports reading values from the value history. The output of the `thread filter` command can be used with the `thread apply` command. For instance, the following evaluates the expression `element % 2 == 1` for each lane of the current thread:

```
(gdb) thread filter :* element % 2 == 1
Filtered Threads:
$1 = "2.129:[1 3 5 7]"
```

The following executes the `print element` on the history value `$1`:

```
(gdb) thread apply $1 print element

Thread 2.129:1 (ZE 0.1.0.0 lane 1):
$2 = 109

Thread 2.129:3 (ZE 0.1.0.0 lane 3):
$3 = 111

Thread 2.129:5 (ZE 0.1.0.0 lane 5):
$4 = 113

Thread 2.129:7 (ZE 0.1.0.0 lane 7):
$5 = 115
```

Likewise, user defined convenience variables are supported:

```
(gdb) print $dummy = "2.129:1"
$6 = "2.129:1"
(gdb) thread apply $dummy print element

Thread 2.129:1 (ZE 0.0.0.0 lane 1):
$11 = 109
```

Filtering Information for One or More Threads/Lanes (Thread Filter)

Command for inspecting threads with the specific information. For example sometimes it is desired to inspect threads for a particular workitem or to inspect threads with some specific variable value. For such needs, the `thread filter` command can be used. For instance, the following evaluates the expression `element % 2 == 0` for the SIMD lanes 3-5 of Thread 2.5 and prints the Thread 2.129 with lane information for each successfully evaluated lane:

```
(gdb) thread filter 2.129:3-5 element % 2 == 0
Filtered Threads:
$1 = "2.129:4"
```

Similarly, the following runs the same command in the context of SIMD lane 3, 5, and 6 of the current thread:

```
(gdb) thread filter :3 :4 :5 element % 2 == 0
Filtered Threads:
$2 = "2.129:4"
```

Stepping GPU Code After a Breakpoint

To stop inside the kernel that is offloaded to the GPU, simply define a breakpoint at a source line inside the kernel. When a GPU thread hits that source line, the debugger stops the execution and shows the breakpoint hit. To single-step a thread over a source-line, use the `step` or `next` commands. The `step` command steps into functions while `next` steps over calls. Before stepping, we recommend to set `scheduler-locking step` to prevent intervention of other threads.

Building a SYCL Executable for Use with Intel® Distribution for GDB*

Much like when you want to debug a host application, you need to set some additional flags to create a binary that can be debugged on the GPU. See [Get Started with Intel® Distribution for GDB* on Linux* Host](#) for details.

For a smooth debug experience when using the just-in-time (JIT) compilation flow, enable debug information emission from the compiler via the `-g` flag, and disable optimizations via the `-O0` flag for both a host and JIT-compiled kernel of the application. The flags for the kernel are taken during link time. For example:

- Compile your program using: `icpx -fsycl -g -O0 -c myprogram.cpp`
- Link your program using: `icpx -fsycl -g -O0 myprogram.o`

If you are using CMake to configure the build of your program, use the `Debug` type for the `CMAKE_BUILD_TYPE`, and append `-O0` to the `CMAKE_CXX_FLAGS_DEBUG` variable. For example: `set (CMAKE_CXX_FLAGS_DEBUG "${CMAKE_CXX_FLAGS_DEBUG} -O0")`

Applications that are built for debugging may take a little longer to start up than when built with the usual “release” level of optimization. Thus, your program may appear to run a little more slowly when started in the debugger. If this causes problems, developers of larger applications may want to use ahead-of-time (AOT) compilation to JIT the offload code when their program is built, rather than when it is run (warning, this may also take longer to build when using `-g -O0`). For more information, see [Compilation Flow Overview](#).

Building an OpenMP* Executable for use with Intel® Distribution for GDB*

Compile and link your program using the `-g -O0` flags. For example:

```
icpx -fopenmp -O0 -fopenmp-targets=spir64 -c -g myprogram.cpp
icpx -fopenmp -O0 -fopenmp-targets=spir64 -g myprogram.o
```

Debugging a Kernel on CPU and GPU: Major Differences

The behavior and commands of Intel® Distribution for GDB* are very close to the upstream version of GDB, i.e. the standard installation you might be familiar with in the operating system distribution you are using. The CPU debugging experience is almost the same; however, there are differences in GPU debugging, coming from the architecture.

In comparison to debugging on a CPU, debugging a kernel on a GPU has a few differences:

Aspect	Description	CPU	GPU
Threads and single instruction, multiple data (SIMD) lanes	When the code is vectorized, <i>threads</i> process vectors of data elements in parallel. <i>SIMD lane</i> is a logical unit of execution for accessing data elements within an instruction.	No SIMD lane context or lane-switching is available for CPU threads.	The debugger supports threads and SIMD lanes. You can switch the context to a particular thread or SIMD lane during debugging. When kernels are compiled in debug mode, most SIMD instructions have 8, 16, or 32 lanes (i.e.

Aspect	Description	CPU	GPU
			execution channels). To refer to a particular SIMD lane of a thread, use a SIMD lane identifier in the format <i>thread ID:lane</i> . To learn more about debugging programs with multiple threads, refer to Chapter 4.10 of Intel® Distribution for GDB* User Guide .

The following commands behave differently during GPU debugging:

Command	Description	Modification	Example/Comment
<code>disassemble</code>	Disassemble the current function.	GEN instructions and registers are shown.	N/A
<code>thread</code>	- Switch to a specified thread. - Switch context to the SIMD lane of the specified thread.	SIMD lanes are supported.	<code>thread 2.5</code> <code>thread 2.6:1</code>
<code>thread apply</code>	Apply a command to the specified SIMD lane of the thread.	SIMD lanes are supported.	<code>thread apply 2.3:* print element</code> prints the value of the <code>element</code> variable for each lane of thread 2.3. Useful for inspecting vectorized values.
<code>info threads</code>	Display information about threads with ID, including their active SIMD lanes.	SIMD lanes are supported.	Use the <code>-stopped</code> option to filter out the unavailable threads.
<code>info devices</code>	Display information about the available devices.	New command.	N/A
<code>commands</code>	Specify a list of commands to execute when your program stops due to a particular breakpoint.	SIMD lanes are supported. With the <code>/</code> a modifier, breakpoint actions apply to all SIMD lanes that match the condition of the specified breakpoint.	<code>commands / a print element end</code> prints the element value for all SIMD lanes that match the condition of the specified breakpoint.
<code>break</code>	Create a breakpoint at a specified line.	- SIMD lanes are supported. You can create a breakpoint at a specific lane.	<code>break 56 thread 2 :3</code> triggers the break only for thread 2 if lane 3 is active. <code>break 56 inferior 2</code> triggers the break only for inferior 2.

Hard-to-find and Useful Information for gdb-oneapi

This page contains links to key pieces of information that are otherwise hard to find.

Useful debugging features

Here are four useful debugging features: live-attach, auto-attach, pretty-printing, and prettify-frames. The first two are used to attach the gdb-oneapi debugger to an application that is already running but is not being debugged. The second two provide ways to simplify the display of complex SYCL objects and templates.

Auto-Attach

The auto-attach feature enables listening to debug events from the GPU.

This feature allows the debugger to launch a `gdbserver-ze` process to listen to GPU debug events and connects the `gdbserver-ze` to the debugger. For each device on the system, an inferior is created on the `gdbserver-ze` connection. This feature is designed to improve your debugging experience and ensure that you can debug the kernel offloaded to GPU.

The auto-attach feature is enabled by default.

The feature does not affect the debugging capability on the CPU device. However, to eliminate the extra output this feature creates, you can turn it off with the `INTELGT_AUTO_ATTACH_DISABLE` environment variable. To do it, execute the following command on the shell before starting `gdb-oneapi`:

```
export INTELGT_AUTO_ATTACH_DISABLE=1
```

To enable the feature again:

```
unset INTELGT_AUTO_ATTACH_DISABLE
```

Reducing Overhead

Executing the `info threads` command may take a noticeable amount of time to complete, because it needs to fetch the data of a large number of threads when debugging GPUs. To reduce the overhead, it is recommended to run the `info threads` command without printing the frame arguments. This can be achieved by changing the setting globally using

```
set print frame-arguments none
```

or by using

```
with print frame-arguments none -- info threads
```

for a single command execution.

Pretty-Printing

The pretty-printing feature simplifies the display of complex objects. If a pretty-printer is registered for the type of value you are going to print, the debugger simplifies the output. Otherwise, the debugger prints the value normally.

Intel® Distribution for GDB* supports pretty-printing for the SYCL* types `accessor`, `buffer`, `device`, `exception`, `handler`, `id`, `item`, `local_accessor`, `queue` and `range` from the `sycl` namespace.

You can write your own pretty-printer for any type. Refer to [Writing a Pretty Printer](#) for more information.

To display the list of pretty-printers available, run the following command:

```
info pretty-print
```

Example output:

```
global pretty-printers:
builtin
  workitem
libsycl
  sycl::_Vl::accessor
  sycl::_Vl::buffer
  sycl::_Vl::device
  sycl::_Vl::exception
  sycl::_Vl::handler
  sycl::_Vl::id
  sycl::_Vl::item
  sycl::_Vl::local_accessor
  sycl::_Vl::queue
  sycl::_Vl::range
```

Pretty-printing is enabled by default. For example, when you print a value of the `sycl::id<2> wiID` variable:

```
print wiID
```

The output is the following:

```
$1 = sycl::id = {29, 16}
```

To disable pretty-printing and display raw content instead, use the `/r` flag:

```
print /r wiID
```

Example output:

```
$2 = {<sycl::_Vl::detail::array<2>> = {common_array = {29, 16}}, static dimensions = <optimized out>}
```

To disable all pretty-printers, use the following command:

```
disable pretty-printer
```

To enable pretty-printers:

```
enable pretty-printer
```

Prettify Frames

Some C++ templates/SYCL constructs make it difficult to view the output of `info threads` and `backtrace` given multi-line function names.

You can use the concept of frame filters to change the visibility of a printed frame with the 'backtrace' command. For details refer to the [GDB Documentation Frame Filter API](#).

It is also possible to change the visibility of a printed frame globally. Consider the setting `print frame-info`:

```
(gdb) set print frame-info source-line
(gdb) info threads -stopped
  Id          Target Id      Frame
<...>
  2.1:[0-15]   ZE 0.0.0.0     53      size_t id0 = GetDim(index, 0);
* 2.9:[*0 1-15] ZE 0.0.1.0     53      size_t id0 = GetDim(index, 0);
  2.33:[0-15]  ZE 0.0.4.0     53      size_t id0 = GetDim(index, 0);
  2.41:[0-15]  ZE 0.0.5.0     53      size_t id0 = GetDim(index, 0);
<...>
```

Refer to the [GDB documentation](#) for more information.

Print settings for kernel data

Given the sample program `array-transform.cpp`:

```

18 using namespace std;
19 using namespace sycl;
20 [...]
26 int main(int argc, char *argv[]) {
27     constexpr size_t length = 64;
28     int input[length];
29     int output[length];
30
31     // Initialize the input
32     for (int i = 0; i < length; i++)
33         input[i] = i + 100;
34
35     try {
36         queue q(default_selector_v, dpc_common::exception_handler);
37         [...]
43         range data_range{length};
44         buffer buffer_in{input, data_range};
45         buffer buffer_out{output, data_range};
46
47         q.submit([&](auto &h) {
48             accessor in(buffer_in, h, read_only);
49             accessor out(buffer_out, h, write_only);
50
51             // kernel-start
52             h.parallel_for(data_range, [=](id<1> index) {
53                 size_t id0 = GetDim(index, 0);
54                 int element = in[index]; // breakpoint-here

```

Inside the kernel we need to use a `sycl::accessor` to access data in the host's `sycl::buffer`. To review the contents of the `sycl::buffer` object `buffer_in` of length 64 from inside the kernel, one can simply print the `sycl::accessor` object in whose pretty-printer provides easy access to the data:

```

(gdb) p in
$1 = sycl::accessor read range 64 = {100, 101, 102, 103, 104, 105, 106,
107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120,
121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134,
135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148,
149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162,
163}

```

Use the print setting `print elements` to further configure number of printed elements:

```

(gdb) set print elements 10
(gdb) p in
$2 = sycl::accessor read range 64 = {100, 101, 102, 103, 104, 105, 106,
107, 108, 109...}

```

Consider the print setting `print repeats` in case the output contains repeated elements:

```

(gdb) show print repeats
Threshold for repeated print elements is 10.

```

Debugging with Source Code Support on Ubuntu

On Ubuntu, `debuginfod` automatically provides **debug symbols** for installed packages. However, due to Debian/Ubuntu packaging limitations, `debuginfod` **cannot serve source code**. Debian-based builds use randomized chroot paths (e.g. `/build/RANDOM-ID/package-1.0/`) that are embedded in the DWARF debug information, and no `debugsource` package is generated. As a result, GDB cannot locate source files automatically.

To enable source-level debugging of Intel GPU driver libraries on Ubuntu, you must manually install debug symbol packages, download the corresponding source code, and configure GDB to map the build paths to your local source tree.

Prerequisites

- Ubuntu 24.04 LTS or Ubuntu 25.10
- Kobuk PPA configured (see [Kobuk intel-graphics PPA on Launchpad](#))
- Approximately 100–150 MB of disk space for debug symbol packages
- Approximately 90 MB of disk space for source code

Setup Debug Symbols and Source Code

1. Install debug symbols for the component you want to debug:

```
# For Level Zero library
sudo apt-get install libze-intel-gpu1-dbgsym

# For OpenCL ICD
sudo apt-get install intel-opencl-icd-dbgsym
```

2. Download and extract the source code for the component:

```
cd /tmp

# For Level Zero library
apt-get source libze-intel-gpu1

# For OpenCL ICD
apt-get source intel-opencl-icd
```

3. Determine the build path recorded in the DWARF debug information. This is the randomized path used during the package build:

```
readelf -wi /usr/lib/x86_64-linux-gnu/libze_intel_gpu.so.1 \
| grep DW_AT_comp_dir
```

Example output:

```
DW_AT_comp_dir : /build/intel-compute-runtime-abcXYZ/intel-compute-runtime-25.05.37020.3
```

4. Configure GDB to map that build path to your local source tree. Add the following to `~/.gdbinit`, replacing the build path with the value obtained in the previous step:

```
# Map the randomized build path to the local source directory
set substitute-path /build/intel-compute-runtime-abcXYZ/intel-compute-runtime-25.05.37020.3 /tmp/
intel-compute-runtime-25.05.37020.3
```

5. Verify it works:

```
gdb-oneapi /usr/lib/x86_64-linux-gnu/libze_intel_gpu.so.1
(gdb) list zeContextCreate
```

You should see the actual source code.

Available Debug Symbol Packages

- `libze-intel-gpu1-dbgsym` — Intel Level Zero library debug symbols
- `intel-opencl-icd-dbgsym` — Intel OpenCL ICD debug symbols

Troubleshooting

- Ensure the Kobuk PPA is properly configured.
- If `list` still shows “No such file or directory”, double-check the `DW_AT_comp_dir` value and your `set substitute-path` mapping.

Unique Features in Intel® Distribution for GDB*

The Intel® Distribution for GDB* (`gdb-oneapi`) is a superset of the GNU Project debugger (`gdb`). `gdb-oneapi` extends a subset of existing GDB commands and features and may, in some cases, add additional debugger commands. You must use `gdb-oneapi` to debug [SYCL C++](#) and [Intel OpenMP](#) workloads (aka kernels) running on Intel GPUs. The additional features in `gdb-oneapi` focus on debugging SYCL and OpenMP applications written in the C, C++, and Fortran languages. The `gdb-oneapi` debugger also contains additional bug fixes and supports pretty-printing for several key SYCL types.

See the Intel® Distribution for GDB* [release notes](#) for details regarding new features, bug fixes, and known issues.

NOTE The `gdb-oneapi` debugger does not support debugging OpenCL kernels running on a GPU. Debugging an OpenCL kernel on an Intel CPU is supported.

NOTE When using `gdb-oneapi` to debug an application on an Intel CPU you may find additional GDB extensions that support CPU instructions and features that are unique to Intel CPUs.

To see which version of GDB the `gdb-oneapi` debugger is based on, type the following:

```
$ gdb-oneapi --version
GNU gdb (Intel(R) Distribution for GDB\* 2024.0.0) 13.1
```

In this example, `gdb-oneapi` is based on version 13.1 of the GNU Project debugger (GDB).

Some Key Extensions in Intel® Distribution for GDB*

- View SIMD lanes; switch between lanes and threads; and view variables and device info when debugging GPU kernel code.
- Use the Visual Studio and Visual Studio Code (aka VSCode) IDE extensions for `gdb-oneapi`.
- Utilize debugger features such as source stepping, expression evaluation, inspecting the call stack, defining conditions for breakpoints, non-stop mode, changing the scheduler-locking setting, and switching the current thread/lane when debugging a GPU kernel.
- Disassemble Intel GPU machine code and display it as Intel GPU assembly language.
- Attach `gdb-oneapi` to an already-running GPU workload.
- Debug the host application code as well as the GPU code in the same debug session.
- Debug multiple GPUs in the same debug session.
- Use convenience variables to inspect GPU work items and global/local indices.
- Create inferior-specific breakpoints that hit only on the inferior that you specify.
- Print Intel® Processor Trace (Intel® PT) PTWRITE payloads in the instruction history and function-call history (supported on Intel CPUs only).
- Read and write Intel® Advanced Matrix Extensions (Intel® AMX) registers (on supported Intel CPUs only).

Modified commands included in Intel® Distribution for GDB*

For additional details regarding `gdb-oneapi` commands, refer to [Differences of debugging a kernel on a CPU and a GPU](#).

Command	Description
info threads [-stopped]	SIMD lane support - Prints additional information about SIMD lanes Allows filtering to show only the stopped threads
thread [thread-id:lane]	SIMD lane support – Allows switching the context to a specific SIMD lane within a thread
thread apply [thread-id[:lane]] all all-lanes [-ascending]] [command]	SIMD lane support - the command can apply a command also to all/ selected active SIMD lanes. Only available threads are applied. To apply all threads including unavailable, add the -unavailable option.
thread filter [thread-id[:lane]] all all-lanes [-ascending]] [-location] [expression]	SIMD lane support - filters from all/selected active SIMD lanes using the location and the expression options. Filtering is applied only on available threads. To filter from all threads including unavailable, add the -unavailable option. For the “all” and the “all-lanes” range selection option, by default the list of filtered thread ids is printed in descending order. To print in ascending order, add the “-ascending” option.
commands [command-list] end	SIMD lane support - additional ‘/a’ modifier.
record function-call-history	Now shows auxiliary information which can be omitted by ‘/s’ modifier”
break [location] inferior[inferior-num] [if [condition]]	Sets an inferior specific breakpoint

New commands included in Intel® Distribution for GDB*

1.	info devices	Display information about available GPU devices
1.	set/show scheduler-locking	Modify run behavior of GPU threads. The setting can be used as following: • set scheduler-locking continue – Scheduler locking for continuing commands during normal execution. • set scheduler-locking eval – Scheduler locking for function calls during normal execution. • set scheduler-locking off – [Shortcut] No locking (threads may preempt at any time). • set scheduler-locking on – [Shortcut] Full locking (no thread except the current thread may run).

1. <code>maint jit dump [addr] [filename]</code>	• set scheduler-locking replay – Scheduler locking settings for replay mode. W/o arguments completely locks the scheduler in replay mode. • set scheduler-locking step – Scheduler locking for stepping commands. W/o arguments locks the scheduler for stepping. Output a JIT object with specific address to the desired file for offline inspection
--	--

Intel® Distribution for GDB* FAQ

What is the difference between `gdb` and `gdb-oneapi`?

The Intel® Distribution for GDB* (`gdb-oneapi`) is a superset of the GNU Project debugger (`gdb`). `gdb-oneapi` extends a subset of existing `gdb` commands and features and may, in some cases, add additional debugger commands. You must use `gdb-oneapi` to debug [SYCL* C++](#) and [Intel OpenMP](#) workloads (aka kernels) running on Intel GPUs. The additional features in `gdb-oneapi` focus on debugging SYCL* and OpenMP applications written in the C, C++, and Fortran languages.

- [Unique Features in Intel® Distribution for GDB*](#)

To see which version of `gdb` the `gdb-oneapi` debugger is based on, run the following command in a terminal session:

```
$ gdb-oneapi --version
GNU gdb (Intel(R) Distribution for GDB* 2024.2.0) 14.2
```

In this example, `gdb-oneapi` is based on version 13.1 of the GNU Project debugger (`gdb`).

If you get a “`gdb-oneapi: command not found`” or “`gdb-oneapi is not recognized`” message, confirm that you have installed the [Intel® oneAPI Toolkit](#) and that you have [setup the Intel oneAPI environment variables](#) in your terminal session.

How do I get started debugging my GPU application?

On your Linux* system, use `gdb-oneapi` to debug CPU and Intel GPU code. On your Windows* system, select the option to install the oneAPI extensions for Visual Studio when you are installing the [Intel® oneAPI Toolkit](#). When debugging a Windows application the Microsoft debugger will be used for debugging CPU code and `gdb-oneapi` will be used for Intel GPU debugging. In both cases (Linux and Windows) the oneAPI Kit include the Intel SYCL compilers and the `gdb-oneapi` debugger. On Windows, oneAPI Kit also include the Intel oneAPI extensions for Visual Studio.

- [Get Started with Intel® Distribution for GDB* on Windows* OS Host](#)
- [Get Started with Intel® Distribution for GDB* on Linux* OS Host](#)

For help debugging GPU-enabled Intel OpenMP applications see this article:

- [Debugging the SYCL and OpenMP* Offload Process](#)

How can I tell if my system software and GPU hardware are properly configured for debugging my application on an Intel GPU?

The simplest way to check that a compatible GPU and drivers are installed is to use the `sycl-ls` command. At a minimum, you need to install the [Intel oneAPI DPC++/C++ compiler](#) to get a copy of the `sycl-ls` utility. For more utility, install all or part of the [Intel® oneAPI Toolkit](#) onto your development system. It includes the Intel oneAPI DPC++/C++ compiler, multiple SYCL-enabled libraries and the `sycl-ls` application. Once installed, open a new terminal session and source the script(s) needed to setup the oneAPI development environment variables. From that same terminal session, enter `sycl-ls` at the prompt, after environment setup has been completed.

If you are using any recommended IDE, such as Visual Studio for Windows, or VS Code for Linux, you can run Intel® Distribution for GDB* Health Check. This tool assists in diagnosing common setup issues and misconfigurations that can prevent successful debugging of applications on an Intel GPU. See the [Running Intel® GDB Debugger Health Check in the Windows Debug Getting Started Guide](#) and [Debugger Health Checks for oneAPI](#) for more details.

See the [oneAPI Development Environmental Setup](#) for details regarding initialization of the oneAPI development environment.

If you are working with a remote Windows debug system and cannot install the oneAPI development tools onto that remote Windows system, you can:

- Copy the `sycl-ls.exe` application from your host Windows development system to a temporary directory on the remote Windows debug system.
- Install the version of the [Intel® oneAPI DPC++/C++ Compiler Runtime for Windows](#) that matches the version of the compiler on your host Windows development system, onto the remote Windows debug system.

If no compatible GPU devices are installed, no GPU devices will appear in the output, as shown in the example below:

```
[openc1:cpu:1] Intel(R) OpenCL, Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz OpenCL 3.0 (Build 0)
[2023.16.8.0.24_160000]
```

If an Intel GPU is installed and the appropriate GPU driver is installed, the GPU device will appear in the output, as shown below:

```
[level_zero:gpu][level_zero:0] Intel(R) oneAPI Unified Runtime over Level-Zero, Intel(R) Arc(TM)
A380 Graphics 12.56.5 [1.5.30714]
[level_zero:gpu][level_zero:1] Intel(R) oneAPI Unified Runtime over Level-Zero, Intel(R) UHD
Graphics 770 12.2.0 [1.5.30714]
[openc1:gpu][openc1:0] Intel(R) OpenCL Graphics, Intel(R) Arc(TM) A380 Graphics OpenCL 3.0 NEO
[32.0.101.6083]
[openc1:gpu][openc1:1] Intel(R) OpenCL Graphics, Intel(R) UHD Graphics 770 OpenCL 3.0 NEO
[32.0.101.6078]
[openc1:cpu][openc1:2] Intel(R) OpenCL, 13th Gen Intel(R) Core(TM) i5-13500 OpenCL 3.0 (Build 0)
[2024.18.9.0.12_160000]
```

Notice the reference to a `level_zero:gpu` device in the very last line of the second set of output from the `sycl-ls` command; this represents a GPU that can be used for running and debugging Intel GPU SYCL code. You might see a GPU device listed in the `sycl-ls` output as `openc1:gpu:0`. An application that runs on an OpenCL GPU device cannot be debugged using `gdb-oneapi`. However, an OpenCL application that runs on the FPGA emulator or on a CPU can be debugged with `gdb-oneapi`. For more help understanding the output of the `sycl-ls` command see the [SYCL Device Selection in the Host Code](#) article in the Intel oneAPI Programming Guide.

In addition to the presence of a GPU, there are environment variables and kernel parameters that must be configured to enable debugging on the GPU. On Windows, the GPU debug setup is handled automatically by the Visual Studio extension for oneAPI development and debug. See the [prerequisites in the Windows Debug Getting Started Guide](#) for more details.

The following environment variable is typically required for a GPU debug session on Linux:

```
export ZET_ENABLE_PROGRAM_DEBUGGING=1
```

If you debug as a “normal” user on a Linux system, you need to be a member of the **video** and **render** groups. Also, the debug feature of the kernel mode driver must be [enabled in the GPU kernel mode driver](#).

The following environment variable can be used to force your SYCL application to run on a GPU device:

```
export ONEAPI_DEVICE_SELECTOR=level_zero:gpu
```

The following environment variables may be required, in addition, for an OpenMP GPU debug session:

```
export LIBOMPTARGET_DEVICE_TYPE=GPU
export LIBOMPTARGET_PLUGIN=LEVEL0
export OMP_TARGET_OFFLOAD=mandatory
```

Help debugging Intel® MPI Library applications can be found in the [GPU Support section](#) of the [Intel® MPI Library Developer Reference for Linux* OS](#) document.

See the links below for more help configuring environment variables and driver parameters for debugging.

- [Enable GPU Debugging in the Kernel Mode Driver](#)
- [Debug Environment Variables](#)
- [Exhaustive List of DPC++ and SYCL Devices Environment Variables](#)
- [Get Started with Intel® Distribution for GDB* on Windows* OS Host](#)
- [Get Started with Intel® Distribution for GDB* on Linux* OS Host](#)

The following links may be helpful for understanding and resolving GPU driver issues:

- [Intel® Distribution for GDB* System Requirements](#)
- [Configuring a Linux System for GPU Debugging](#)
- [Linux Kernel Driver Types for Debugging](#)
- [Released GPU Drivers for Linux](#)
- [Intel Arc & Iris Xe Graphics Drivers for Windows](#)
- [List of Intel GPU PCI IDs](#)

Where do I download current GPU drivers for debugging my application on GPU?

- [Linux Kernel Driver Types for Debugging](#)
- [Released GPU Drivers for Linux](#)
- [Intel Arc & Iris Xe Graphics Drivers for Windows](#)

Can I debug my GPU application with Eclipse?

Yes, the Eclipse IDE for C/C++ Developers is supported “natively” by the `gdb-oneapi` debugger on Linux (Windows is not supported). This means that standard debug support is provided using the Eclipse IDE for C/C++ Developers distribution (Eclipse version 2023-06 (4.28) or later).

For access to advanced `gdb-oneapi` features (such as visualizing SIMD lanes) in an IDE, you must use Visual Studio (on Windows) or Visual Studio Code (on Linux). The guide below describes how to configure the Eclipse IDE to use `gdb-oneapi` to debug applications on Linux.

- [Using Eclipse IDE to Debug SYCL Applications on a Linux OS Host](#)
- [SSH: Running Applications Built with Eclipse*](#)
- [SSH: Password-less Access to Remote Linux* Target](#)

Can I debug my GPU application with Visual Studio on Windows?

Yes, a `gdb-oneapi` Visual Studio extension is available for use with the Microsoft Visual Studio IDE. During installation of the [Intel® oneAPI Toolkit](#) you are given the option to install a collection of Visual Studio extensions that support development of Intel oneAPI applications on your Windows system.

- The list of supported GPU devices [can be found here](#).
- Be sure to [install the latest GPU drivers](#) onto your Windows debug system.
- Local GPU debugging requires a dedicated GPU for display, in addition to an Intel GPU that supports SYCL or Intel OpenMP compute applications.
- Use remote GPU debugging when your Windows test system does not have a dedicated GPU for display. In that case, two Windows systems are required: a host and a target (the remote system). Visual Studio and the [Intel® oneAPI Toolkit](#) must be installed on the host system. During development, your oneAPI application is deployed from the host system to the target system, where it will run and be debugged.
- Your remote target system may require installation of the oneAPI runtime libraries to run your application on the remote system. You can find the runtime packages in the “Runtime Versions” section of the [“Single Component Downloads and Runtime Versions”](#) article.
- For a step-by-step guide, read [Get Started with Intel® Distribution for GDB* on Windows* OS Host](#).

Can I debug my GPU application with Visual Studio Code?

Yes, several oneAPI extensions are available for developing oneAPI applications with Visual Studio Code (aka VSCode). A summary of the Intel oneAPI extensions for VSCode can be found in the [Using Visual Studio Code with Intel® oneAPI Toolkit User Guide](#). You can [find the latest oneAPI extensions for VSCode in the VSCode marketplace](#), specifically the extensions that show the “oneAPI” logo. Or type “oneapi” into the extension marketplace dialog inside a running instance of VSCode (Ctrl-Shift-x or “gear icon > Extensions”).

NOTE: The VSCode `gdb-oneapi` extension only supports debugging your application on Linux systems.

To debug, you need the “[GDB with GPU Debug Support for Intel® oneAPI Toolkit](#)” extension as well as the `gdb-oneapi` debugger, which can be found in the [Intel® oneAPI Toolkit](#).

The “[Code Sample Browser for Intel® Software Developer Tools](#)” extension is a convenient way to browse and install Intel oneAPI samples that are ready to be compiled and run. Most of the samples are short and easy to understand. They are especially useful if you want to familiarize yourself with the Intel oneAPI development tools.

- GDB with GPU Debug Toolkit <<https://marketplace.visualstudio.com/items?itemName=intel-corporation.oneapi-gdb-debug>>` ____
- [Debugging the DPC++ debugger using Visual Studio® Code on Ubuntu](#)
- [Remote Development using SSH](#)
- [Remote SSH: Tips and Tricks](#)
- [SSH: Password-less Access to Remote Linux Targets](#)
- [Using Visual Studio Code with Intel oneAPI Toolkits](#)

Where are the `gdb-oneapi` release notes?

- [Intel® Distribution for GDB* Release Notes](#)

How do I enable source-level debugging on Ubuntu?

On Ubuntu, `debuginfod` provides debug symbols automatically but cannot serve source code due to Debian/Ubuntu packaging limitations. To enable source-level debugging of Intel GPU driver libraries, you must manually install debug symbol packages, download the source code, and configure GDB path mappings. See [Debugging with Source Code Support on Ubuntu](#) for step-by-step instructions.

Where is the `gdb-oneapi` debugging troubleshooting guide?

- [Linux Debugging Troubleshooting Guide](#)
- [Windows Debugging Troubleshooting Guide](#)

Where is the Intel® Distribution for GDB* user manual?

The Intel® Distribution for GDB* (`gdb-oneapi`) is a superset of the GNU Project debugger (`gdb`). The complete [Intel® Distribution for GDB* User Manual](#) (PDF) is a superset of the [Debugging with GDB](#) documentation; that is, the `gdb-oneapi` documentation extends the existing `gdb` documentation. An [Intel® Distribution for GDB* Cheat Sheet](#) (PDF) is a one-sheet summary of the most useful `gdb-oneapi` commands.

Switch Between a CPU Context and a GPU Context When Debugging with VSCode*

When `gdb-oneapi` hits a GPU breakpoint, you view the context of that GPU thread, i.e., the local variables in that thread. If you need to “go back” and inspect some variables in a CPU thread, you need to scroll to the top of the “CALL STACK” and locate the “main” thread.

Before scrolling through the threads in the CALL STACK window you should make note of the GPU thread that you are leaving, if you want to return to the current GPU thread. See the relationship, below, of the various thread identifiers when viewing the SIMD Lanes:

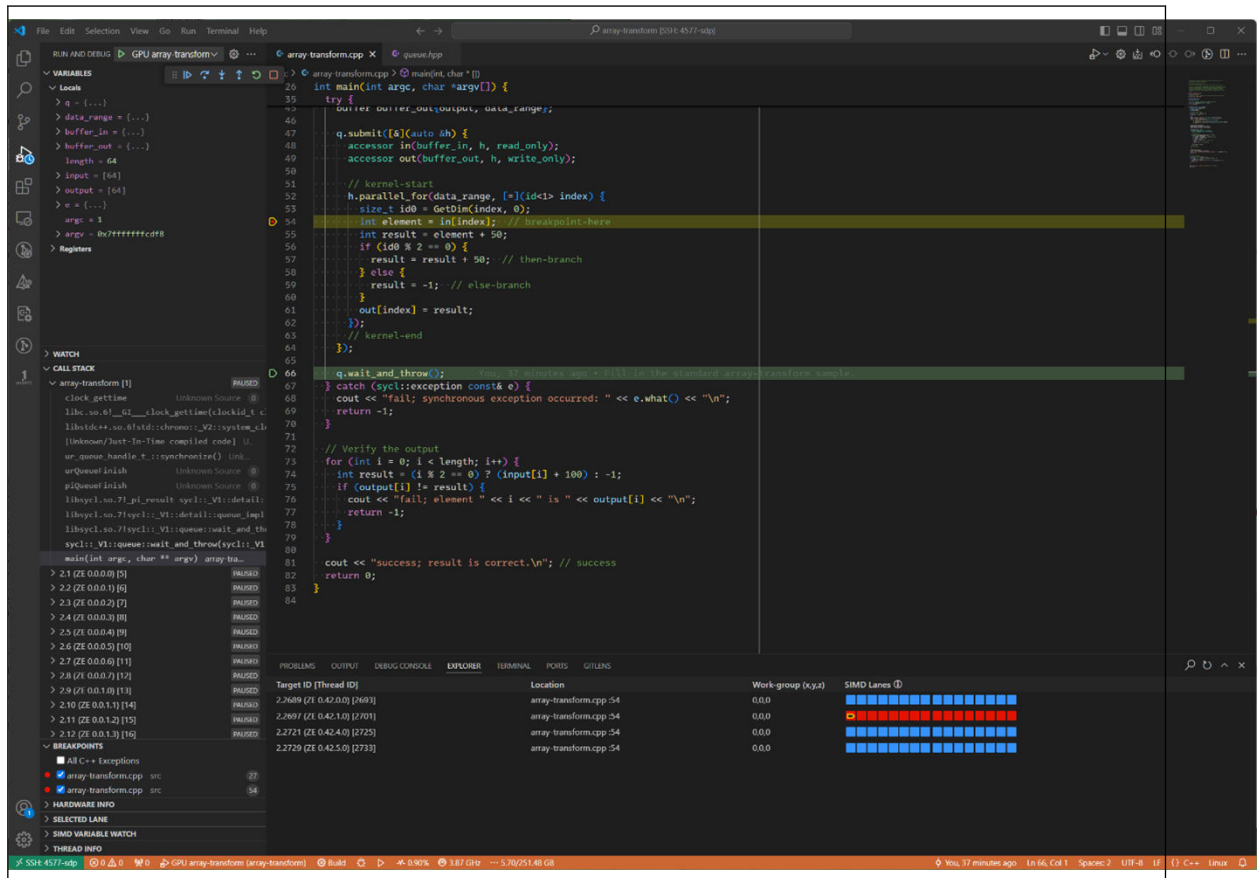
Relationship of the various thread identifiers in the SIMD Lanes

The screenshot shows the VS Code interface with the Intel Distribution for GDB (gdb-oneapi) extension. The main editor displays the source code of 'array-transform.cpp'. The left sidebar shows the 'CALL STACK' window with a list of threads. The bottom right shows the 'SIMD Lanes' window, which displays a grid of lanes for different threads. The 'SIMD Lanes' window is currently showing the 'main' thread (Thread ID: 2.2698) and its lanes are highlighted in blue. The 'SIMD Lanes' window also shows the 'Work-group (x,y,z)' and 'SIMD Lanes (i)' for each thread.

Thread ID	Location	Work-group (x,y,z)	SIMD Lanes (i)
2.2698 (ZE 0.42.0.0) [2698]	array-transform.cpp:54	0,0,0	0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99,100,101,102,103,104,105,106,107,108,109,110,111,112,113,114,115,116,117,118,119,120,121,122,123,124,125,126,127,128,129,130,131,132,133,134,135,136,137,138,139,140,141,142,143,144,145,146,147,148,149,150,151,152,153,154,155,156,157,158,159,160,161,162,163,164,165,166,167,168,169,170,171,172,173,174,175,176,177,178,179,180,181,182,183,184,185,186,187,188,189,190,191,192,193,194,195,196,197,198,199,200,201,202,203,204,205,206,207,208,209,210,211,212,213,214,215,216,217,218,219,220,221,222,223,224,225,226,227,228,229,230,231,232,233,234,235,236,237,238,239,240,241,242,243,244,245,246,247,248,249,250,251,252,253,254,255,256,257,258,259,260,261,262,263,264,265,266,267,268,269,270,271,272,273,274,275,276,277,278,279,280,281,282,283,284,285,286,287,288,289,290,291,292,293,294,295,296,297,298,299,300,301,302,303,304,305,306,307,308,309,310,311,312,313,314,315,316,317,318,319,320,321,322,323,324,325,326,327,328,329,330,331,332,333,334,335,336,337,338,339,340,341,342,343,344,345,346,347,348,349,350,351,352,353,354,355,356,357,358,359,360,361,362,363,364,365,366,367,368,369,370,371,372,373,374,375,376,377,378,379,380,381,382,383,384,385,386,387,388,389,390,391,392,393,394,395,396,397,398,399,400,401,402,403,404,405,406,407,408,409,410,411,412,413,414,415,416,417,418,419,420,421,422,423,424,425,426,427,428,429,430,431,432,433,434,435,436,437,438,439,440,441,442,443,444,445,446,447,448,449,450,451,452,453,454,455,456,457,458,459,460,461,462,463,464,465,466,467,468,469,470,471,472,473,474,475,476,477,478,479,480,481,482,483,484,485,486,487,488,489,490,491,492,493,494,495,496,497,498,499,500,501,502,503,504,505,506,507,508,509,510,511,512,513,514,515,516,517,518,519,520,521,522,523,524,525,526,527,528,529,530,531,532,533,534,535,536,537,538,539,540,541,542,543,544,545,546,547,548,549,550,551,552,553,554,555,556,557,558,559,560,561,562,563,564,565,566,567,568,569,570,571,572,573,574,575,576,577,578,579,580,581,582,583,584,585,586,587,588,589,590,591,592,593,594,595,596,597,598,599,600,601,602,603,604,605,606,607,608,609,610,611,612,613,614,615,616,617,618,619,620,621,622,623,624,625,626,627,628,629,630,631,632,633,634,635,636,637,638,639,640,641,642,643,644,645,646,647,648,649,650,651,652,653,654,655,656,657,658,659,660,661,662,663,664,665,666,667,668,669,670,671,672,673,674,675,676,677,678,679,680,681,682,683,684,685,686,687,688,689,690,691,692,693,694,695,696,697,698,699,700,701,702,703,704,705,706,707,708,709,710,711,712,713,714,715,716,717,718,719,720,721,722,723,724,725,726,727,728,729,730,731,732,733,734,735,736,737,738,739,740,741,742,743,744,745,746,747,748,749,750,751,752,753,754,755,756,757,758,759,760,761,762,763,764,765,766,767,768,769,770,771,772,773,774,775,776,777,778,779,780,781,782,783,784,785,786,787,788,789,790,791,792,793,794,795,796,797,798,799,800,801,802,803,804,805,806,807,808,809,810,811,812,813,814,815,816,817,818,819,820,821,822,823,824,825,826,827,828,829,830,831,832,833,834,835,836,837,838,839,840,841,842,843,844,845,846,847,848,849,850,851,852,853,854,855,856,857,858,859,860,861,862,863,864,865,866,867,868,869,870,871,872,873,874,875,876,877,878,879,880,881,882,883,884,885,886,887,888,889,890,891,892,893,894,895,896,897,898,899,900,901,902,903,904,905,906,907,908,909,910,911,912,913,914,915,916,917,918,919,920,921,922,923,924,925,926,927,928,929,930,931,932,933,934,935,936,937,938,939,940,941,942,943,944,945,946,947,948,949,950,951,952,953,954,955,956,957,958,959,960,961,962,963,964,965,966,967,968,969,970,971,972,973,974,975,976,977,978,979,980,981,982,983,984,985,986,987,988,989,990,991,992,993,994,995,996,997,998,999,1000,1001,1002,1003,1004,1005,1006,1007,1008,1009,1010,1011,1012,1013,1014,1015,1016,1017,1018,1019,1020,1021,1022,1023,1024,1025,1026,1027,1028,1029,1030,1031,1032,1033,1034,1035,1036,1037,1038,1039,1040,1041,1042,1043,1044,1045,1046,1047,1048,1049,1050,1051,1052,1053,1054,1055,1056,1057,1058,1059,1060,1061,1062,1063,1064,1065,1066,1067,1068,1069,1070,1071,1072,1073,1074,1075,1076,1077,1078,1079,1080,1081,1082,1083,1084,1085,1086,1087,1088,1089,1090,1091,1092,1093,1094,1095,1096,1097,1098,1099,1100,1101,1102,1103,1104,1105,1106,1107,1108,1109,1110,1111,1112,1113,1114,1115,1116,1117,1118,1119,1120,1121,1122,1123,1124,1125,1126,1127,1128,1129,1130,1131,1132,1133,1134,1135,1136,1137,1138,1139,1140,1141,1142,1143,1144,1145,1146,1147,1148,1149,1150,1151,1152,1153,1154,1155,1156,1157,1158,1159,1160,1161,1162,1163,1164,1165,1166,1167,1168,1169,1170,1171,1172,1173,1174,1175,1176,1177,1178,1179,1180,1181,1182,1183,1184,1185,1186,1187,1188,1189,1190,1191,1192,1193,1194,1195,1196,1197,1198,1199,1200,1201,1202,1203,1204,1205,1206,1207,1208,1209,1210,1211,1212,1213,1214,1215,1216,1217,1218,1219,1220,1221,1222,1223,1224,1225,1226,1227,1228,1229,1230,1231,1232,1233,1234,1235,1236,1237,1238,1239,1240,1241,1242,1243,1244,1245,1246,1247,1248,1249,1250,1251,1252,1253,1254,1255,1256,1257,1258,1259,1260,1261,1262,1263,1264,1265,1266,1267,1268,1269,1270,1271,1272,1273,1274,1275,1276,1277,1278,1279,1280,1281,1282,1283,1284,1285,1286,1287,1288,1289,1290,1291,1292,1293,1294,1295,1296,1297,1298,1299,1300,1301,1302,1303,1304,1305,1306,1307,1308,1309,1310,1311,1312,1313,1314,1315,1316,1317,1318,1319,1320,1321,1322,1323,1324,1325,1326,1327,1328,1329,1330,1331,1332,1333,1334,1335,1336,1337,1338,1339,1340,1341,1342,1343,1344,1345,1346,1347,1348,1349,1350,1351,1352,1353,1354,1355,1356,1357,1358,1359,1360,1361,1362,1363,1364,1365,1366,1367,1368,1369,1370,1371,1372,1373,1374,1375,1376,1377,1378,1379,1380,1381,1382,1383,1384,1385,1386,1387,1388,1389,1390,1391,1392,1393,1394,1395,1396,1397,1398,1399,1400,1401,1402,1403,1404,1405,1406,1407,1408,1409,1410,1411,1412,1413,1414,1415,1416,1417,1418,1419,1420,1421,1422,1423,1424,1425,1426,1427,1428,1429,1430,1431,1432,1433,1434,1435,1436,1437,1438,1439,1440,1441,1442,1443,1444,1445,1446,1447,1448,1449,1450,1451,1452,1453,1454,1455,1456,1457,1458,1459,1460,1461,1462,1463,1464,1465,1466,1467,1468,1469,1470,1471,1472,1473,1474,1475,1476,1477,1478,1479,1480,1481,1482,1483,1484,1485,1486,1487,1488,1489,1490,1491,1492,1493,1494,1495,1496,1497,1498,1499,1500,1501,1502,1503,1504,1505,1506,1507,1508,1509,1510,1511,1512,1513,1514,1515,1516,1517,1518,1519,1520,1521,1522,1523,1524,1525,1526,1527,1528,1529,1530,1531,1532,1533,1534,1535,1536,1537,1538,1539,1540,1541,1542,1543,1544,1545,1546,1547,1548,1549,1550,1551,1552,1553,1554,1555,1556,1557,1558,1559,1560,1561,1562,1563,1564,1565,1566,1567,1568,1569,1570,1571,1572,1573,1574,1575,1576,1577,1578,1579,1580,1581,1582,1583,1584,1585,1586,1587,1588,1589,1590,1591,1592,1593,1594,1595,1596,1597,1598,1599,1600,1601,1602,1603,1604,1605,1606,1607,1608,1609,1610,1611,1612,1613,1614,1615,1616,1617,1618,1619,1620,1621,1622,1623,1624,1625,1626,1627,1628,1629,1630,1631,1632,1633,1634,1635,1636,1637,1638,1639,1640,1641,1642,1643,1644,1645,1646,1647,1648,1649,1650,1651,1652,1653,1654,1655,1656,1657,1658,1659,1660,1661,1662,1663,1664,1665,1666,1667,1668,1669,1670,1671,1672,1673,1674,1675,1676,1677,1678,1679,1680,1681,1682,1683,1684,1685,1686,1687,1688,1689,1690,1691,1692,1693,1694,1695,1696,1697,1698,1699,1700,1701,1702,1703,1704,1705,1706,1707,1708,1709,1710,1711,1712,1713,1714,1715,1716,1717,1718,1719,1720,1721,1722,1723,1724,1725,1726,1727,1728,1729,1730,1731,1732,1733,1734,1735,1736,1737,1738,1739,1740,1741,1742,1743,1744,1745,1746,1747,1748,1749,1750,1751,1752,1753,1754,1755,1756,1757,1758,1759,1760,1761,1762,1763,1764,1765,1766,1767,1768,1769,1770,1771,1772,1773,1774,1775,1776,1777,1778,1779,1780,1781,1782,1783,1784,1785,1786,1787,1788,1789,1790,1791,1792,1793,1794,1795,1796,1797,1798,1799,1800,1801,1802,1803,1804,1805,1806,1807,1808,1809,1810,1811,1812,1813,1814,1815,1816,1817,1818,1819,1820,1821,1822,1823,1824,1825,1826,1827,1828,1829,1830,1831,1832,1833,1834,1835,1836,1837,1838,1839,1840,1841,1842,1843,1844,1845,1846,1847,1848,1849,1850,1851,1852,1853,1854,1855,1856,1857,1858,1859,1860,1861,1862,1863,1864,1865,1866,1867,1868,1869,1870,1871,1872,1873,1874,1875,1876,1877,1878,1879,1880,1881,1882,1883,1884,1885,1886,1887,1888,1889,1890,1891,1892,1893,1894,1895,1896,1897,1898,1899,1900,1901,1902,1903,1904,1905,1906,1907,1908,1909,1910,1911,1912,1913,1914,1915,1916,1917,1918,1919,1920,1921,1922,1923,1924,1925,1926,1927,1928,1929,1930,1931,1932,1933,1934,1935,1936,1937,1938,1939,1940,1941,1942,1943,1944,1945,1946,1947,1948,1949,1950,1951,1952,1953,1954,1955,1956,1957,1958,1959,1960,1961,1962,1963,1964,1965,1966,1967,1968,1969,1970,1971,1972,1973,1974,1975,1976,1977,1978,1979,1980,1981,1982,1983,1984,1985,1986,1987,1988,1989,1990,1991,1992,1993,1994,1995,1996,1997,1998,1999,2000,2001,2002,2003,2004,2005,2006,2007,2008,2009,2010,2011,2012,2013,2014,2015,2016,2017,2018,2019,2020,2021,2022,2023,2024,2025,2026,2027,2028,2029,2030,2031,2032,2033,2034,2035,2036,2037,2038,2039,2040,2041,2042,2043,2044,2045,2046,2047,2048,2049,2050,2051,2052,2053,2054,2055,2056,2057,2058,2059,2060,2061,2062,2063,2064,2065,2066,2067,2068,2069,2070,2071,2072,2073,2074,2075,2076,2077,2078,2079,2080,2081,2082,2083,2084,2085,2086,2087,2088,2089,2090,2091,2092,2093,2094,2095,2096,2097,2098,2099,2100,2101,2102,2103,2104,2105,2106,2107,2108,2109,2110,2111,2112,2113,2114,2115,2116,2117,2118,2119,2120,2121,2122,2123,2124,2125,2126,2127,2128,2129,2130,2131,2132,2133,2134,2135,2136,2137,2138,2139,2140,2141,2142,2143,2144,2145,2146,2147,2148,2149,2150,2151,2152,2153,2154,2155,2156,2157,2158,2159,2160,2161,2162,2163,2164,2165,2166,2167,2168,2169,2170,2171,2172,2173,2174,2175,2176,2177,2178,2179,2180,2181,2182,2183,2184,2185,2186,2187,2188,2189,2190,2191,2192,2193,2194,2195,2196,2197,2198,2199,2200,2201,2202,2203,2204,2205,2206,2207,2208,2209,2210,2211,2212,2213,2214,2215,2216,2217,2218,2219,2220,2221,2222,2223,2224,2225,2226,2227,2228,2229,2230,2231,2232,2233,2234,2235,2236,2237,2238,2239,2240,2241,2242,2243,2244,2245,2246,2247,2248,2249,2250,2251,2252,2253,2254,2255,2256,2257,2258,2259,2260,2261,2262,2263,2264,2265,2266,2267,2268,2269,2270,2271,2272,2273,2274,2275,2276,2277,2278,2279,2280,2281,2282,2283,2284,2285,2286,2287,2288,2289,2290,2291,2292,2293,2294,2295,2296,2297,2298,2299,2300,2301,2302,2303,2304,2305,2306,2307,2308,2309,2310,2311,2312,2313,2314,2315,2316,2317,2318,2319,2320,2321,2322,2323,2324,2325,2326,2327,2328,2329,2330,2331,2332,2333,2334,2335,2336,2337,2338,2339,2340,2341,2342,2343,2344,2345,2346,2347,2348,2349,2350,2351,2352,2353,2354,2355,2356,2357,2358,2359,2360,2361,2362,2363,2364,2365,2366,2367,2368,2369,2370,2371,2372,2373,2374,2375,2376,2377,2378,2379,2380,2381,2382,2383,2384,2385,2386,2387,2388,2389,2390,2391,2392,2393,2394,2395,2396,2397,2398,2399,2400,2401,2402,2403,2404,2405,2406,2407,2408,2409,2410,2411,2412,2413,2414,2415,2416,2417,2418,2419,2420,2421,2422,2423,2424,2425,2426,2427,2428,2429,2430,2431,2432,2433,2434,2435,2436,2437,2438,2439,2440,2441,2442,2443,2444,2445,2446,2447,2448,2449,2450,2451,2452,2453,2454,2455,2456,2457,2458,2459,2460,2461,2462,2463,2464,246

When you scroll to the top of the CALL STACK, you'll find the "main" thread's current context. Note that the "main" thread only runs on the CPU, not on a GPU. That main thread will look something like the following, which shows a collection of local variables that are present in that CPU thread:

Collection of local variables present in a CPU thread



If you need to get back to the GPU thread that hit the GPU breakpoint, you will have to scroll back down to the numbered thread that you saved, before scrolling up to the CPU thread. There may be many unused GPU execution units, especially when running a GPU kernel that operates on relatively small arrays and matrices.

Videos: Intel® Distribution for GDB*

[Intel® Distribution for GDB* Easily Debugs Multiple GPU Applications at Once](#)

This video shows how to use the Intel® Distribution for GDB* to debug multiple applications running on Intel® architecture GPUs.

[MPI Multi-GPU Device Offload Debug Using the Intel® Distribution for GDB*](#)

This video shows you how to debug MPI (Message Passing Interface) programs using Intel® Distribution of GDB*.

[Debugging Just-in-Time and Ahead-of-Time Compiled GPU Code with the Intel® Distribution for GDB* Part One](#)

This is the first of two videos that explain the differences of Just-in-Time and Ahead-of-Time compilation and new compiling accelerated code for your target platform.

Debugging Just-in-Time and
Ahead-of-Time Compiled GPU
Code with the Intel®
Distribution for GDB* Part
Two

This is the second of two videos that explain the differences of Just-in-Time and Ahead-of-Time compilation and new compiling accelerated code for your target platform.